# WILSON ZHENG

(413) 429-6350 | wilsonz@andrew.cmu.edu | WilsonZheng0327.github.io

## EDUCATION

**Carnegie Mellon University (CMU)** *Pittsburgh, PA*
Master of Science in Artificial Intelligence and Innovation, GPA: 4.1/4.3 Aug 2025 – May 2027

**The University of Virginia (UVA)** *Charlottesville, VA*
Bachelor's Degree in Computer Science & Physics, GPA: 3.91/4.0 Aug 2021 – May 2025
Echols Scholar, College Science Scholar, Dean's List

Relevant Courses: LLM Systems, Computer Architecture, Compilers, Operating Systems, Cloud Computing

## WORK EXPERIENCE

***Software Engineering Intern*, Google,** Sunnyvale, CA May 2026 – Aug 2026

- Engineered a virtual fault-injection framework in **C++20** for security controllers, simulating instruction-skip attacks to uncover secure-boot bypass vulnerabilities
- Authored hardened security patches mitigating glitch attacks via compiler-aware anti-glitch loop checks and reinforced verification primitives
- Debugged bare-metal firmware and analyzed raw Arm disassembly to confirm both vulnerabilities and their fixes
- Extended existing test suites to exhaustively evaluate Secure Boot verification logic under fault-injection scenarios

***Software Development Engineer Intern,* Amazon Web Services,** Beijing, China Jun 2025 – Aug 2025

- Engineered **Amazon Q**-powered AI unit test generation tool, raising code coverage to up to 90% across services
- Integrated into Amazon's CI/CD pipeline, enabling automatic test generation and saving ~10 hr/wk of manual testing
- Designed event-driven, scalable architecture on **AWS** to support asynchronous test generation workflows
- Developed multi-stage validation pipeline to detect and filter LLM hallucinations, significantly improving test reliability by eliminating invalid test cases before integration

## OPEN SOURCE & PROJECTS

***SGLang-Omni – Contributor, PersonaPlex integration lead*** June 2026 – Present

PersonaPlex-7B full-duplex speech-to-speech serving

- Integrated NVIDIA's **PersonaPlex-7B** into SGLang-Omni: Mimi codec, a 7B temporal transformer on SGLang's paged KV cache, and the depth transformer, with greedy parity against NVIDIA's reference
- Co-designed the model-agnostic full-duplex session runtime and used it to enable live PersonaPlex calls: the model listens and speaks at the same time in 80 ms frames, with no VAD or turn boundaries, and keeps up in real time over 10-minute calls on one GPU
- Led the optimization roadmap with ~10 contributors: **10× throughput** at concurrency 16 and **74%** lower p95 TTFA
- Key optimizations: cross-request batching, CUDA-graph replay for the streaming Mimi decoder (**5.6×** faster decode step, **22%** lower TTFA) and the 7B backbone decode (**9%** lower latency), and a **6.6×** faster encoder→LM handoff

Fun-ASR-Nano serving optimization

- Built a pre-LM audio encoder service that runs full-context encoding before LM admission, with single-flight dedup and an LRU embedding cache: **1.3× cold / 3.3× warm** throughput at H100 concurrency 32
- Profiled with Nsight Systems and built a bucketed CUDA-graph runner for the launch-bound 70-layer SANM encoder, delivering **2.3× throughput** (56→130 samples/s) at concurrency 32 with no accuracy regression

***NodeTorch – Open-source node-based visual ML model builder*** Apr 2026 – Present

- Built a drag-and-drop visual ML builder for composing and training neural networks without code, on a registry-driven engine that separates the graph runtime, type system and ML domain, so new nodes, datasets and training paradigms plug in without engine changes
- Built interactive per-layer visualizations (forward activations, parameter distributions, attention patterns, loss surfaces) so learners see *why* a model works, not just that it works
- Integrated an in-app AI assistant with a backend agent loop that lets any LLM operate the app

***High-Performance CUDA Kernel for DeepSeek Sparse Attention on NVIDIA B200*** Feb 2026 – Apr 2026

- Optimized a BF16 **CUDA** kernel on NVIDIA B200 for DSA, achieving 22–50× speedup over the PyTorch reference across 23 trace workloads using WMMA m16n16k16 tensor cores, double-buffered cp.async KV gather, and a Split-K + online-softmax reduce pipeline that eliminated the small-batch SM occupancy bottleneck

## SKILLS

Programming Languages: Python, C++, C, C#, TypeScript, SQL, Bash, x86, CUDA

Frameworks and Tools: PyTorch, HuggingFace, OpenCV, Nsight Systems, SGLang, GCP, AWS, Docker